

Bahria University Islamabad, E-8 Campus

Department of Computer Science

Shooting Game with Score Management System

using 8086 Assembly Language

Course: Computer Organization and Assembly Language

Course Code: CEN 323

Instructor: Adnan Jelani

Group Members

Syed Huzaifa Farooq (01-135232-097)

Abdullah Javed (01-135232-003)

Saiyab Waheed (01-135232-085)

Submission Date: May 17, 2026

Contents

1	Abstract	2
2	Introduction	2
3	Problem Statement	2
4	Objectives	2
5	Features	3
6	System Design	3
6.1	Basic Flow	4
7	Implementation Details	4
7.1	Arrays	4
7.2	Macros	4
7.3	Procedures	4
7.4	Interrupts Used	5
7.5	Sorting Algorithm	5
8	Assembly Concepts Used	5
9	CCA Mapping	6
9.1	CCA1: Complex Problem Solving	6
9.2	CCA4: Design Constraints	6
9.3	CCA8: Team Collaboration	6
10	Testing and Results	6
11	Screenshots	7
12	Team Contributions	8
13	Challenges and Lessons Learned	8
14	Conclusion	8
15	References	9

1 Abstract

This project is an interactive Shooting Game with a Score Management System developed using 8086 Assembly Language in EMU8086. The game allows a player to enter their name, shoot randomly generated targets, receive hit or miss feedback, and earn a score based on correct shots. The system also stores player names and scores, displays a leaderboard, sorts scores in descending order, and provides grade-based feedback. The project demonstrates important assembly language concepts such as arrays, loops, procedures, macros, stack usage, interrupts, arithmetic operations, conditional jumps, and indexed addressing. The final outcome is a menu-driven game that combines entertainment with low-level programming concepts.

2 Introduction

Assembly language provides a close view of how computer hardware works internally. This project was developed to apply Computer Organization and Assembly Language concepts in a practical and interactive way. Instead of creating a simple arithmetic or array-based program, this project combines game logic with score management.

The system includes player input, random target generation, hit/miss checking, score calculation, leaderboard management, and sorting.

3 Problem Statement

Basic assembly programs often only demonstrate simple calculations or loops. This project solves the problem of creating a more complex and interactive 8086 assembly program by combining multiple modules such as game logic, user input, score tracking, animations, and leaderboard sorting into one complete system.

4 Objectives

- To develop an interactive shooting game in 8086 Assembly Language.
- To implement player name and score storage using arrays.
- To generate random targets using system timer interrupt.
- To implement hit and miss logic using conditional jumps.
- To sort leaderboard scores using Bubble Sort.
- To use procedures and macros for modular programming.

- To demonstrate stack usage and register preservation.

5 Features

- Menu-driven interface
- Player name input
- Random target generation
- Hit and miss detection
- Score tracking
- Score bar visualization
- Streak bonus
- Grade calculation
- Leaderboard display
- Descending score sorting
- Sound effect using beep
- ASCII-based game visuals

6 System Design

The project is divided into several modules:

- **Menu Module:** Displays options to play game, view leaderboard, or exit.
- **Game Module:** Handles target generation, player input, hit/miss logic, and score updates.
- **Animation Module:** Displays gun, target, hit, miss, and countdown visuals.
- **Score Module:** Stores scores and player names in arrays.
- **Leaderboard Module:** Displays stored player records.
- **Sorting Module:** Sorts scores in descending order using Bubble Sort.

6.1 Basic Flow

1. Program starts and displays welcome screen.
2. User selects option from menu.
3. If play game is selected, user enters name.
4. Random target is generated.
5. User enters shot number.
6. Program checks hit or miss.
7. Score is updated.
8. Final score is saved.
9. Leaderboard displays sorted records.

7 Implementation Details

7.1 Arrays

Arrays are used to store player names and scores:

- `scores db 10 dup(0)`
- `names db 200 dup(0)`

7.2 Macros

The project uses macros such as:

- `NEWLINE` for printing a new line
- `BEEP` for sound effect

7.3 Procedures

The project uses procedures for modularity:

- `draw_bar`
- `draw_gun`
- `draw_target`

- `anim_hit`
- `anim_miss`
- `show_countdown`
- `sort_scores`

7.4 Interrupts Used

- **INT 21H:** Used for input and output operations.
- **INT 10H:** Used for screen display handling.
- **INT 1AH:** Used for generating random-like target values using system timer.

7.5 Sorting Algorithm

Bubble Sort is used to sort scores in descending order. When scores are swapped, the corresponding player names are also swapped to keep the leaderboard accurate.

8 Assembly Concepts Used

- Data definition using DB
- Arrays
- Arithmetic operations
- Conditional jumps
- Loops
- Procedures
- Macros
- Stack usage
- Indexed addressing using SI and DI
- Interrupt handling
- Local variable usage using BP offset

9 CCA Mapping

9.1 CCA1: Complex Problem Solving

The project solves a non-trivial problem by combining multiple modules such as gameplay, score tracking, leaderboard sorting, animations, and grading. The program contains non-linear control flow through menu selection, conditional jumps, random target generation, and hit/miss decisions.

9.2 CCA4: Design Constraints

The project is implemented completely in 8086 Assembly Language using EMU8086. It works within limitations of 8086 registers, 16-bit addressing, segmented memory, and DOS/BIOS interrupts. Data is handled using arrays and registers instead of high-level data structures.

9.3 CCA8: Team Collaboration

The project was divided into modules among group members. Each member contributed to a specific part such as game logic, leaderboard, UI/animations, testing, and documentation. GitHub was used to maintain version history and track contributions.

10 Testing and Results

Test Case	Expected Result	Actual Result
Select Play Game	Game starts and asks for name	Game starts correctly
Enter correct target number	HIT message and score increase	HIT displayed and score updated
Enter wrong number	MISS message shown	MISS displayed correctly
Finish 5 rounds	Final score displayed and saved	Score saved successfully
Open Leaderboard	Sorted records shown	Leaderboard displayed correctly
Select Exit	Program terminates	Program exits successfully

11 Screenshots

Upload your screenshots in Overleaf with these exact names: `menu.png`, `gameplay.png`, and `leaderboard.png`.

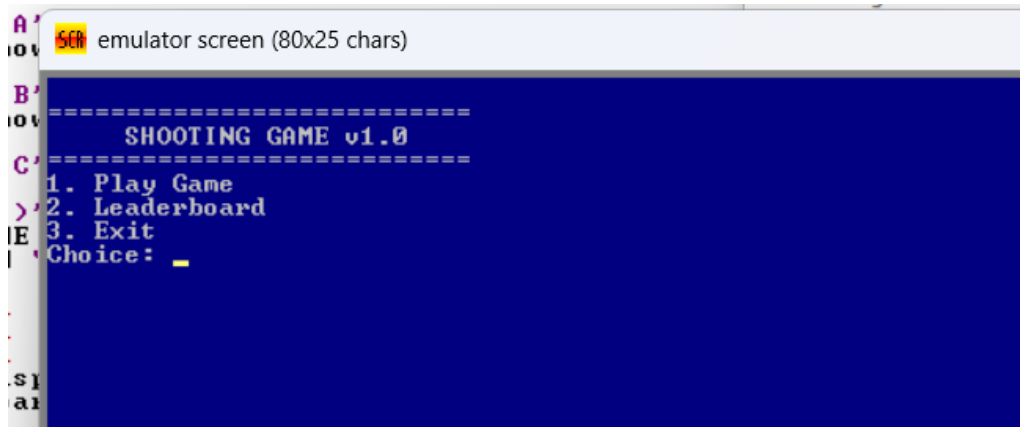


Figure 1: Main Menu Screen

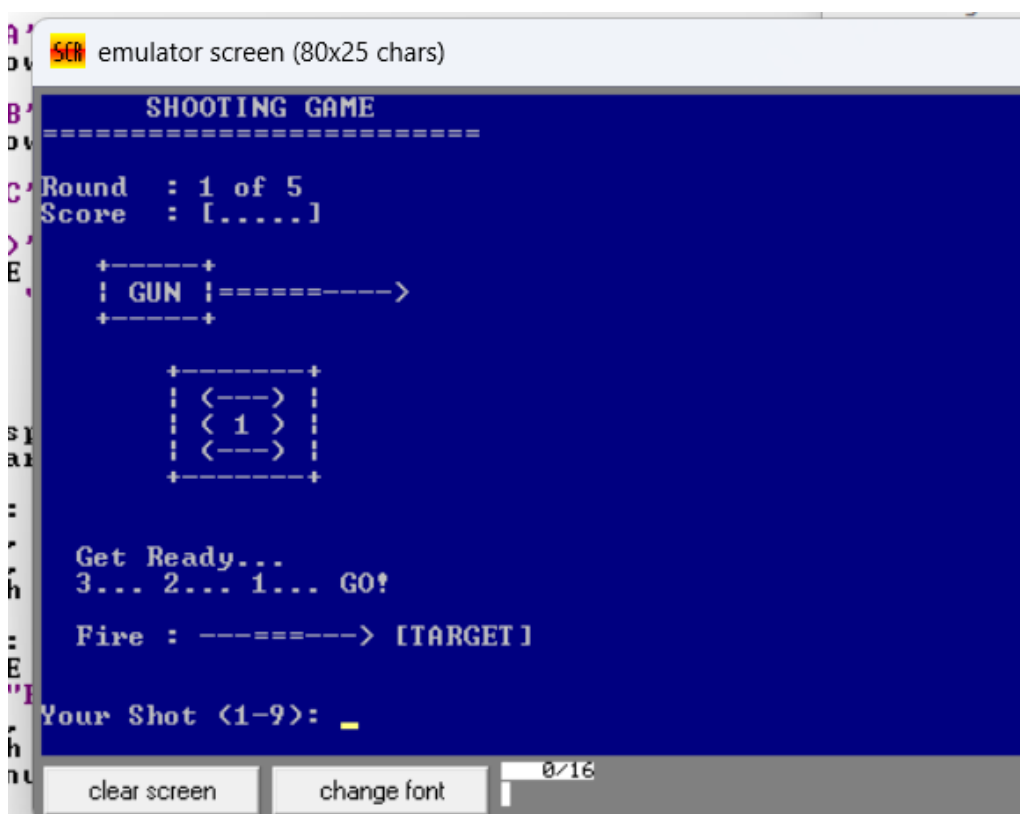


Figure 2: Gameplay Screen

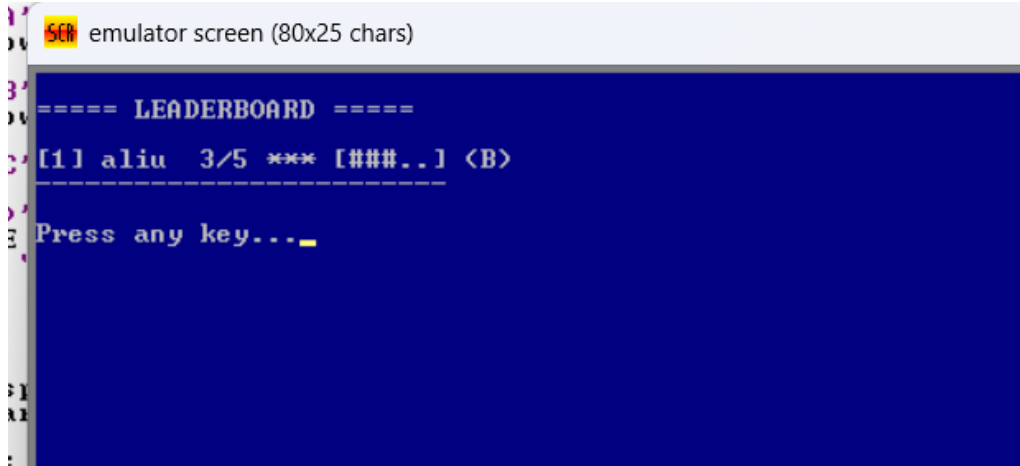


Figure 3: Leaderboard Screen

12 Team Contributions

Name	Registration Number	Module(s) Owned
Syed Huzaifa Farooq	01-135232-097	Game logic and testing
Abdullah Javed	01-135232-003	Leaderboard and sorting module
Saiyab Waheed	01-135232-085	UI, animations, report, and integration

13 Challenges and Lessons Learned

During development, several technical challenges were faced. The most common issue was register conflict, especially when using CX in loops and procedures. Another challenge was sorting player names along with scores so that leaderboard records remained accurate. Some BIOS interrupt functions were not fully supported in EMU8086, so safer alternatives were used.

AI-assisted tools were consulted for debugging support, syntax correction, and understanding assembly concepts, but the implementation, testing, and integration were manually verified by the team.

14 Conclusion

This project successfully demonstrates how 8086 Assembly Language can be used to build an interactive game with score management. It combines multiple assembly concepts including arrays, loops, procedures, macros, stack usage, interrupts, sorting, and

conditional logic. The project improved our understanding of low-level programming, register handling, memory access, and modular assembly design.

Future improvements may include more advanced animations, difficulty levels, and persistent score storage.

15 References

The following resources were consulted during the development and understanding of this project:

- EMU8086 Official Documentation and Examples.
- Intel 8086 Assembly Language Instruction Set Manual.
- DOS Interrupt Reference for INT 21H, INT 10H, and INT 1AH.
- Course Lab Manuals for Computer Organization and Assembly Language.
- Online educational tutorials related to 8086 Assembly Language programming.
- GitHub repositories and open-source Assembly Language examples used for learning and comparison purposes.
- Stack Overflow discussions for debugging and assembly syntax understanding.
- AI-assisted learning tools including ChatGPT and Claude AI were consulted for concept explanation, debugging guidance, and improving code structure.
- GitHub Repository: <https://github.com/Saiyab-Waheed/Interactive-Shooting-Game-System>.